Incubator Temperature Controller Plan

Sep 27, 2026 · @Lauren

The chamber temperature is controlled by a PI controller tuned with lambda tuning (λ = 300 s, Kp = 0.0155, Ti = 594 s) and setpoint weighting b = 0.5, which reaches 37 °C in about 40 min with no overshoot in simulation.

1. The plan in one page

The goal is a controller that measures the chamber temperature every second, compares it to 37 °C, and sets the heater duty cycle between 0 and 1.

```
37°C → [compare] → error → [PI controller] → duty (0 to 1) → [chamber] → temperature
          ↑                                                                  |
          └──────────────────────────────────── sensor ──────────────────────┘
```

The order to work in:

- [ ] Keep the single-node chamber model as it is, and add one sentence saying the time constant is approximate (section 2).
- [ ] Design a PI controller with lambda tuning, λ = 300 s (section 3).
- [ ] Add setpoint weighting b = 0.5 to stop overshoot (section 4).
- [ ] Show it is stable with the pole locations and phase margin (section 5).
- [ ] Build the chamber and controller in Simulink (section 6).
- [ ] Run the four tests and save the plots (section 7).
- [ ] Once the chamber is built, do a step test and plug the real numbers into the same formulas (section 8).

The key idea to hold on to: the model does not have to be perfect. It only has to get the controller into the right neighbourhood. The step test on the real chamber gives the final numbers, and the Simulink model stays the same.

2. The chamber model

The chamber behaves like a first-order system: gain K = 128 K per unit duty and time constant τ = 594 s (about 10 min).

The idea in plain words

Think of the chamber as a bucket with holes in it. The water level is the temperature. The heaters are a tap pouring in. The holes are heat leaking out through the walls, the plate and to the glass. The level rises until the leak rate matches the tap, then stays there.

- **Resistance R** is how small the holes are. Bigger R means heat leaks out more slowly.
- **Capacitance C** is how wide the bucket is. Wider means it takes longer to fill. Air alone is tiny (1.2 J/K), but the wall soaks up heat as the air warms, so the wall is included (166 J/K).

The heat balance

Stored heat changes at the rate of heat in minus heat out:

```
C_c \frac{dT_c}{dt} = \mu_h P_h + \frac{T_g - T_c}{R_{cg}} - \frac{T_c - T_a}{R_{ca}} - \frac{T_c - T_a}{R_{cp}}
```

Symbol	Meaning	Value
C_c	Chamber capacitance (air + wall)	167 J/K
P_h	Heater power at full duty	36 W
μ_h	Duty cycle	0 to 1
R_ca	Chamber to ambient through walls	6.09 K/W
R_cg	Chamber to glass	11.9 K/W
R_cp	Chamber to ambient through plate	30.53 K/W
T_g	Glass temperature (own loop)	about 39 °C
T_a	Ambient temperature	about 22 °C

The transfer function

Taking the Laplace transform (in deviation variables) and normalising:

```
T_c(s) = \frac{128\,\mu_h(s) + 0.299\,T_g(s) + 0.701\,T_a(s)}{594s + 1}
```

So the plant the controller sees, from duty to chamber temperature, is:

```
G(s) = \frac{K}{\tau s + 1} = \frac{128}{594s + 1}
```

The glass and ambient terms are disturbances. They push the temperature around, but the controller does not set them.

What the numbers mean

- **K = 128 K:** full heater power would raise the chamber 128 °C above its rest point. That is far more than needed, so holding 37 °C needs only about 8% duty.
- **τ = 594 s:** after about 10 min the chamber is two thirds of the way to its final temperature.
- **No cooling:** if the temperature goes above 37 °C, the only way back down is waiting for heat to leak out. So overshoot is the main thing to avoid.

The one honest caveat

Lumping the wall in with the air is an approximation, because the wall sits between chamber and ambient temperature rather than at chamber temperature. So τ is probably a bit too long (the real value is likely 5 to 10 min). Put this in the report:

> The printed wall stores far more heat than the air, so it is included in the chamber capacitance. This is an approximation, since the wall sits between the chamber and ambient temperatures rather than at the chamber temperature, so the model is expected to overestimate the time constant. The model is used for initial controller tuning, and the time constant and gain will be identified from a step test on the built chamber.

3. The controller: PI with lambda tuning

The controller is a PI (no D term) with Kp = 0.0155 and Ti = 594 s, chosen so the closed loop behaves like a simple first-order system with time constant λ = 300 s.

Why PI and not full PID

- **P** reacts to how far off you are right now.
- **I** adds up the error over time. It is what makes you actually reach 37 °C, because holding 37 °C needs about 8% duty and P alone cannot provide that when the error is zero.
- **D** reacts to how fast the temperature is changing. Your model is first order, temperature changes slowly, and D mainly amplifies sensor noise. So it is dropped.

Almost every real incubator or oven uses PI, so this is an easy choice to justify.

Lambda tuning

You pick one number, λ: how fast you want the closed loop to respond. Then:

$$T_i = \tau \qquad K_p = \frac{\tau}{K\lambda} \qquad K_i = \frac{K_p}{T_i}$$

With λ = 300 s (about half the plant time constant):

Parameter	Formula	Value
Ti	τ	594 s
Kp	594 / (128 × 300)	0.0155 (1.55% duty per °C)
Ki	Kp / Ti	2.61×10^{-5} per °C·s

Smaller λ is faster but more aggressive and more sensitive to model errors. Larger λ is gentler and slower.

The controller transfer function, step by step

Ki is in there. The same transfer function can be written three ways, and the (594s + 1) version is just the one that makes the maths easy.

Step 1: what the controller does in time. The duty is P times the error, plus I times the error added up over time:

$$u(t) = K_p\,e(t) + K_i \int e(t)\,dt$$

Step 2: take the Laplace transform. Integrating in time becomes dividing by s. So:

$$C(s) = \frac{U(s)}{E(s)} = K_p + \frac{K_i}{s} = 0.0155 + \frac{2.61\times10^{-5}}{s}$$

This is the most natural form. The first term is P, the second is I.

Step 3: put it over one fraction. Multiply Kp by s/s so both terms share the denominator s:

$$C(s) = \frac{K_p\, s + K_i}{s} = \frac{0.0155s + 2.61\times10^{-5}}{s}$$

Step 4: take Ki out as a common factor. Divide the top by Ki and multiply the front by Ki:

$$C(s) = K_i\,\frac{\frac{K_p}{K_i}s + 1}{s} = 2.61\times10^{-5}\;\frac{594s + 1}{s}$$

That is where the 594 comes from: Kp / Ki = 0.0155 / 2.61×10^{-5} = 594 s. This ratio is called the integral time Ti. We deliberately chose Kp and Ki so that it equals τ, the plant time constant. Check by multiplying out: $2.61 \times 10^{-5} \times 594 = 0.0155$, so the top is $0.0155s + 2.61 \times 10^{-5}$, the same as Step 3.

So all three forms are the same controller:

Form	Written out	Useful for
P + I	$0.0155 + 2.61 \times 10^{-5} / s$	Seeing the P and I parts, entering P and I in Simulink
One fraction	$(0.0155s + 2.61 \times 10^{-5}) / s$	Finding the zero and pole
Factored	$2.61 \times 10^{-5} (594s + 1) / s$	Seeing the cancellation below

The controller has a pole at s = 0 (the integrator, the s on the bottom) and a zero at s = −1/594 (where 594s + 1 = 0).

Why it works: pole-zero cancellation

The plant has (594s + 1) on the **bottom**:

```
G(s) = \frac{128}{594s + 1}
```

The controller, in its factored form, has (594s + 1) on the **top**. Multiply them together to get the loop transfer function L(s), and they cancel:

```
L(s) = C(s)\,G(s) = 2.61\times10^{-5}\,\frac{594s + 1}{s}\cdot\frac{128}{594s + 1} = \frac{128 \times 2.61\times10^{-5}}{s} = \frac{0.00334}{s} = \frac{1}{300s}
```

The 300 is λ, which is how Kp was chosen in the first place: 1/0.00334 = 300 s. In words, the controller's zero sits exactly on top of the plant's slow pole and removes it, leaving a pure integrator.

The closed loop

For a unity feedback loop, the closed-loop transfer function is L / (1 + L):

```
\frac{T_c(s)}{T_{sp}(s)} = \frac{L(s)}{1 + L(s)} = \frac{\frac{1}{300s}}{1 + \frac{1}{300s}}
```

Multiply the top and bottom by 300s to clear the fractions:

$$\frac{T_c(s)}{T_{sp}(s)} = \frac{1}{300s + 1}$$

This is a first-order system with time constant λ = 300 s and a gain of 1. What that means:

- **Gain of 1:** set s = 0 and you get 1, so the final temperature equals the setpoint exactly. Zero steady-state error, thanks to the integrator.
- **First order:** a first-order step response rises smoothly and never overshoots.
- **Time constant 300 s:** it covers 63% of a setpoint change in 5 min and is essentially there (98%) after 4λ = 20 min.

The pole in the closed loop is at s = −1/300. It is real and negative, so the loop is stable.

Two things the transfer function does not show

- **The 0 to 1 clamp.** Duty cannot go negative or above 100%. The PI needs **anti-windup**, which stops the integral growing while the output is clamped. Otherwise it overshoots. In Simulink this is one setting (clamping).
- **The warm glass.** With heaters off, the chamber does not rest at 22 °C. The glass warms it to about 27 °C on its own. The cancellation does not cover this, and it causes about 0.3 °C of overshoot on warm-up. Section 4 fixes it.

4. Stopping overshoot: setpoint weighting

Setting b = 0.5 removes the overshoot in every case tested, at the cost of a slower warm-up (about 40 min instead of 30).

Why it overshoots at all

During warm-up the error is large for a long time, so the integral keeps adding up. By the time the chamber reaches 37 °C, the integral is holding more heat than the chamber needs, so it carries on past 37. The warm glass makes this worse by heating the chamber on its own.

The fix

A normal PI puts the full error into the P term. With setpoint weighting, the P term only sees part of the setpoint:

$$u = K_p\,(b\,T_{sp} - T_c) + K_i \int (T_{sp} - T_c)\,dt$$

With b = 0.5, P pushes less hard during warm-up, so the approach is gentler. The I term still uses the full error, so it still reaches exactly 37 °C. Disturbances (like an ambient drop) do not change the setpoint, so they are handled exactly as before.

Important: use temperatures measured **above ambient** (subtract 22 °C from both the setpoint and the measurement before the controller). Setpoint weighting on absolute temperatures gives different answers in °C and K, which is wrong.

Setpoint transfer function

The feedback loop is unchanged, so stability is unchanged. Only the setpoint response changes:

$$\frac{T_c(s)}{T_{sp}(s)} = \frac{b\,\tau s + 1}{(\tau s + 1)(\lambda s + 1)} = \frac{297s + 1}{(594s + 1)(300s + 1)}$$

Same two poles as before, plus a zero that shapes the warm-up.

Simulation results

Overshoot and time to settle within ±0.2 °C, warm-up from 22 °C:

Case	Normal PI (b = 1)	b = 0.5
Nominal model	0.32 °C / 30 min	0.00 °C / 39 min
Capacitance 34% lower (110 J/K)	0.00 °C / 20 min	0.00 °C / 46 min
Capacitance 50% higher (250 J/K)	1.05 °C / 46 min	0.15 °C / 27 min
Heater 30% weaker	0.22 °C / 32 min	0.00 °C / 42 min
Heater 30% stronger	0.36 °C / 27 min	0.00 °C / 38 min
Two-node model (air and wall separate)	0.00 °C / 36 min	0.00 °C / 52 min

An alternative is a setpoint ramp (raise the setpoint from 22 to 37 °C over 20 min). It works when the model is right but is less robust: 0.55 °C overshoot if the capacitance is 50% higher. Setpoint weighting is the better choice.

5. Is it stable?

Yes, with a large margin: both closed-loop poles are real and negative, and the phase margin is 90°.

Check	Result	What it means
Closed-loop poles	$s = -1/300$ and $s = -1/594$	Both real and negative, so stable with no oscillation

Check	Result	What it means
Phase margin	90°	The best possible. L(s) = 1/(300s) has −90° phase everywhere
Delay tolerance	about 470 s (= λ·π/2)	The real lag from sensor and wall will be seconds to tens of seconds
Model errors	Stable for capacitance −34% to +50%, heater ±30%	Still works if the model is off
Two-node model	Stable, no overshoot	The single-node simplification does not hurt the controller

The honest caveats, which the step test in section 8 deals with:

- Disturbances such as an ambient drop recover slowly, at roughly the plant's 10 min time constant. This is a known trade-off of lambda tuning and is fine for an incubator. In simulation, a 4 °C ambient drop dips the chamber by about 0.7 °C before it recovers.
- At about 8% duty, PWM resolution matters. 8-bit PWM gives only about 20 steps at 8%, so use at least 10-bit.
- Add a firmware or hardware cutoff that turns the heaters off above about 40 °C. A stable controller does not protect you from a sensor falling off.

Sampling and delay

Sampling at 1 s adds about 1.5 s of effective delay, which costs only 0.3° of phase margin, so it does not affect the design.

Sampling does not add a derivative term. What it adds is a small delay: the controller holds the duty constant between samples, which acts like a delay of half a sample. Add up to one more sample for reading the sensor and computing, and the worst case is about 1.5 s.

Check	Value	Verdict
Phase lost at crossover (ω = 1/300 rad/s)	1.5/300 rad, about 0.3°	Phase margin goes from 90° to 89.7°
Delay that would cause instability	about 470 s	Sampling delay is about 300 times smaller
Rule of thumb: sample time below λ/10	λ/10 = 30 s vs Ts = 1 s	Comfortably met, even 10 s sampling would be fine
Discretisation method (Euler, Tustin)	Δt/τ = 1/594	All methods give the same result at this ratio

Sampling only matters when the sample time gets close to the system's speed, like a motor with a 50 ms time constant. A 5 to 10 min thermal system is very forgiving.

The delays that matter more are physical ones, and the step test in section 8 catches them as θ:

- **Sensor lag.** A sensor in a housing or glued to a surface may lag by 5 to 30 s. Still small against 300 s, but larger than the sampling delay.
- **Sensor placement.** If the sensor reads the air, the samples in the wells will lag behind it. That is a physical question, not a control one.
- **Quantisation.** The integral changes by only about 2.6×10^{-5} per second for a 1 °C error. Store it as a float, not an integer, or it rounds to zero and never builds up. Sensor resolution (for example 0.0625 °C on a DS18B20) and PWM resolution matter more than the sample rate.

Showing it in Simulink: the discrete PID block at Ts = 1 s already includes sampling. To show delay does not matter, add a **Transport Delay** block on u inside the Chamber subsystem and rerun the warm-up with delays of 0, 1.5 and 30 s. The curves should sit almost on top of each other. That plot plus the phase margin numbers above is the argument for the report.

Discrete-time check (z-domain)

The controller was designed in continuous time and runs digitally at Ts = 1 s, and converting to the z-domain confirms the digital loop is stable with poles almost exactly where the continuous design puts them.

This approach is called **design by emulation**. It is valid when the sample time is much shorter than the loop's time constant; the usual rule is Ts < λ/10 = 30 s, and Ts = 1 s. A full z-domain design is not needed, but this check proves the continuous design carries over.

Plant with zero-order hold (PWM holds the duty for one sample):

$$G(z) = \frac{K(1 - a)}{z - a}, \qquad a = e^{-T_s/\tau} = e^{-1/594} = 0.998318 \quad\Rightarrow\quad G(z) = \frac{0.2153}{z - 0.998318}$$

Controller exactly as coded (forward Euler integral, I += e·dt):

$$C(z) = K_p + \frac{K_i T_s}{z - 1}$$

Closed-loop poles:

Discrete pole	Continuous pole mapped to z = e^(sTs)	Match
0.996672	e^(−1/300) = 0.996672	✓
0.998315	e^(−1/594) = 0.998318	✓

Both poles are inside the unit circle, so the digital loop is stable, and they match the continuous design to five decimal places.

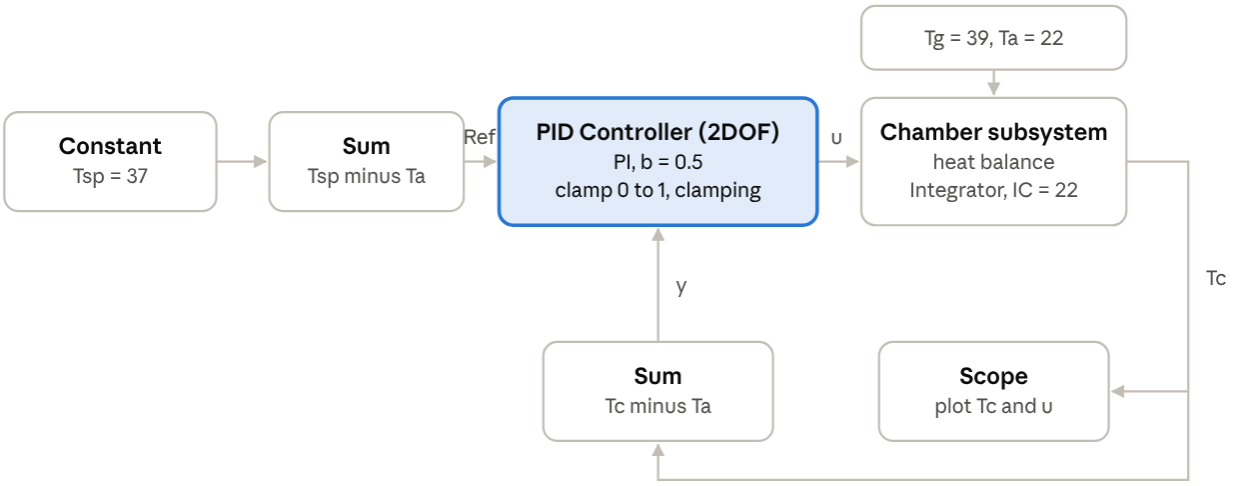
In MATLAB (add to the analysis script):

```
Ts = 1;
Gz = c2d(G, Ts, 'zoh');              % plant with zero-order hold
z  = tf('z', Ts);
Cz = Kp + Ki*Ts/(z - 1);             % controller as coded
Tz = feedback(Cz*Gz, 1);
disp(pole(Tz))                       % should match the table
disp(abs(pole(Tz)))                  % all < 1 -> stable
figure; step(15*Tz, 15*T, 4000);     % discrete vs continuous, should overlap
legend('Discrete (Ts = 1 s)', 'Continuous'); grid on;
```

The overlapping discrete and continuous step responses make a good figure for the report.

6. Building it in Simulink

The model has two parts: a Chamber subsystem built straight from the heat balance, and a 2DOF PID block that is fed temperatures above ambient.



Simulink top level · PI loop around the chamber

The setpoint and the measurement both have ambient subtracted before they reach the controller, and the chamber output goes back round as the measurement.

Step 1: the parameter script

Make a script called `incubator_params.m` and run it before every simulation. Using variable names in the blocks means you can change a number here and rerun without touching the model.

```
% ---- Plant ----
C   = 167;      % chamber capacitance, J/K
Ph  = 36;       % heater power at full duty, W
Rcg = 11.9;     % chamber to glass, K/W
Rca = 6.09;     % chamber to ambient (walls), K/W
Rcp = 30.53;    % chamber to ambient (plate), K/W
Tg  = 39;       % glass temperature, degC
Ta  = 22;       % ambient temperature, degC
T0  = 22;       % starting chamber temperature, degC

% ---- Model numbers (from the transfer function) ----
K   = 128;      % gain, K per unit duty
tau = 594;      % time constant, s

% ---- Controller ----
lambda = 300;               % desired closed-loop time constant, s
Kp  = tau/(K*lambda);       % 0.0155
```

```
Ki  = Kp/tau;               % 2.61e-5
b   = 0.5;                  % setpoint weight
Tsp = 37;                   % setpoint, degC
Ts  = 1;                    % controller sample time, s
```

Step 2: the Chamber subsystem

Build it from the heat balance rather than the transfer function. That way it uses real temperatures (22, 37 °C) and the disturbances are just inputs.

```
\frac{dT_c}{dt} = \frac{1}{C}\left[P_h u + \frac{T_g - T_c}{R_{cg}} - (T_c - T_a)\left(\frac{1}{R_{ca}} + \frac{1}{R_{cp}}\right)\right]
```

All blocks are in the Library Browser (search by name):

1. **Integrator:** its output is Tc. Set Initial condition to `T0`.
2. **Heater path:** u goes into a **Gain** of `Ph`.
3. **Glass path:** a **Constant** `Tg` and Tc go into a **Sum** with signs `+-`, then a **Gain** of `1/Rcg`.
4. **Ambient path:** Tc and a **Step** block (Initial value `Ta`, Final value `Ta` for now) go into a **Sum** with signs `+-`, then a **Gain** of `1/Rca + 1/Rcp`.
5. **Main Sum** with signs `++-`: heater path, glass path, ambient path, in that order.
6. **Gain** of `1/C`, feeding the Integrator.
7. Feed the Integrator output (Tc) back into the glass and ambient Sums.

Select all of these, right-click, choose **Create Subsystem from Selection** and name it Chamber. The input is u and the output is Tc. The top level then looks clean for the report.

Step 3: the controller

1. **Constant** `Tsp` and a **Constant** `Ta` go into a **Sum** with signs `+-`. This goes into the PID's **Ref** input.
2. Tc and `Ta` go into another **Sum** with signs `+-`. This goes into the PID's **y** input.
3. Use the **PID Controller (2DOF)** block. Double-click it and set:
   - Controller: **PI**
   - Form: **Parallel**
   - Time domain: **Discrete-time**, Sample time `Ts` (continuous also works)
   - P: `Kp`, I: `Ki`
   - Setpoint weight b: `b` (c does nothing since there is no D)
   - **Output Saturation** tab: tick Limit output, Upper `1`, Lower `0`
   - Anti-windup method: **clamping**

4. The PID output is u. Connect it to the Chamber input.

Step 4: outputs and settings

- A **Scope** on Tc and another on u.
- **To Workspace** blocks on Tc and u (save format: Timeseries) so you can make clean plots for the report.
- Model settings: Stop time `4000` s (about 67 min). The default solver is fine.

7. Tests that prove the design works

Four simulations, each giving one plot for the report, show the controller reaches 37 °C without overshoot, rejects disturbances, and survives model errors.

Test	What to change	What it proves	What to expect
1. Warm-up	Nothing (T0 = 22, Tsp = 37)	Reaches setpoint, no overshoot	About 40 min to settle, duty starts near 0.23 and settles near 0.077
2. Ambient disturbance	Stop time 7200, Step block Ta goes 22 to 18 at t = 4000	Rejects disturbances	Dips about 0.7 °C, then recovers to 37 °C
3. Robustness	Rerun test 1 with C = 110 and 250, then Ph = 25 and 47	Still works if the model is off	Worst case about 0.15 °C overshoot (C = 250)
4. Choosing λ and b	Overlay λ = 150, 300, 500, and b = 1 vs b = 0.5	Justifies your choices	b = 1 overshoots about 0.3 °C, b = 0.5 does not

Two checks worth pointing out in the report:

- **The steady duty should settle near 7.7%.** That matches the hand calculation from the model, so the simulation and the maths agree.
- **Anti-windup:** at λ = 300 the duty never reaches 100%, so the clamp does not engage on a normal warm-up. It is a safety net. To show it working, run λ = 50 with and without clamping and compare.

A quick way to run the robustness cases from a script instead of editing by hand (change `incubator_model` to your model's name):

```
incubator_params;
figure; hold on;
for Ctest = [110 167 250]
    C = Ctest;
    out = sim('incubator_model');
    plot(out.Tc.Time/60, out.Tc.Data, 'DisplayName', sprintf('C = %d J/K',
Ctest));
end
yline(37, '--', 'Setpoint');
xlabel('Time (min)'); ylabel('Chamber temperature (°C)');
legend show; grid on;
C = 167;   % put it back
```

This assumes your To Workspace block for Tc is named `Tc` and saves as Timeseries.

8. Once the chamber is built

A single open-loop step test gives the real K, τ and delay θ; they go into the same formulas and the Simulink model does not change.

Step 1: run the step test

1. Let the chamber sit with heaters off until its temperature is steady. Keep the glass at its normal temperature (its own loop running) so conditions match real use.
2. Log the chamber temperature, ambient temperature and duty every 1 s.
3. Switch the film heaters to a **fixed** duty, around 15% (no controller). This gives a rise of roughly 15 to 20 °C, big enough to read clearly without going too hot.
4. Keep logging until the temperature stops rising, roughly 40 to 60 min.
5. Also weigh the printed wall, so you can recalculate C from its measured mass.

Step 2: read the numbers off the curve

- **K** = (final temperature − starting temperature) / duty step. For example, a 19 °C rise at 15% duty gives K = 19 / 0.15 = 127.
- **θ (delay)** = the time between switching on and the temperature starting to rise. It may be close to zero.
- **τ** = the time to reach 63% of the total rise, measured from the end of the delay.

If the curve jumps quickly at first and then creeps up slowly (the two-node behaviour), fit τ to the slow part. That part is what matters for the controller.

Optional, if you have the System Identification Toolbox: `sys = tfest(iddata(dT, du, 1), 1, 0, NaN)` fits a first-order model with delay automatically, where dT and du are temperature and duty measured from their starting values.

Step 3: retune

The new model is first order plus dead time:

```
G(s) = \frac{K\,e^{-\theta s}}{\tau s + 1}
```

The lambda tuning formulas become:

```
T_i = \tau \qquad K_p = \frac{\tau}{K(\lambda + \theta)} \qquad K_i = \frac{K_p}{T_i}
```

If θ is near zero this is the same as before. A good rule is to keep λ at least as large as θ, and not much smaller than half of τ.

Step 4: check and close the loop

1. Put the measured K, τ and C into `incubator_params.m`. If there is a delay, add a **Transport Delay** block (delay θ) on u inside the Chamber subsystem.
2. Rerun the Simulink tests from section 7 with the new gains.
3. Plot the measured step response against the model's prediction for the same duty. This is a strong figure for the report.
4. Put the new gains on the microcontroller and run a real warm-up to 37 °C. Compare it to the simulated warm-up.

What changes and what doesn't

Stays the same	Gets updated
PI structure, setpoint weighting b = 0.5, anti-windup, 0 to 1 clamp	K, τ, C from the measurements
Simulink model layout	Kp and Ki (recomputed by the script)
Test plan and stability argument	Possibly λ, if the real delay is large

For the report, the story is: model, lambda tuning, simulated tests, step test, retuned gains, measured vs simulated.